Project Documentation - Behaviour Guardian

A real-time behavioural nudge system that watches *how* a person interacts with a web page and warns them, in the moment, when that interaction pattern looks like someone being rushed or pressured - not when the website itself looks unsafe.

---

1. Problem Statement

Most financial fraud today doesn't happen because someone visited a fake website. It happens on the **real bank login page**, the **real UPI app**, the **real payment gateway** - while the victim is on the phone with a scammer, or being messaged instructions on WhatsApp, telling them exactly what to type and how fast to type it.

The website is genuine. The form is genuine. The only thing that isn't normal is the *person's behaviour in that moment* - and nothing currently watches for that.

2. Existing Problems

Existing tool	What it actually checks	What it misses
URL/domain reputation, Google Safe Browsing, ad blockers	Is this a known-bad site or link?	Nothing - the scammer's instructions run on a completely legitimate site
Antivirus / anti-phishing scanners	Malicious code, known phishing kits	A person calmly typing their own real OTP into a real bank page
Bank-side fraud detection (enterprise behavioural biometrics - BioCatch, NuData/Mastercard, Feedzai, etc.)	Session and device signals, server-side, after the fact	It's invisible to the person, bought by institutions, and reviewed by an analyst later - not a real-time nudge to the individual
OTP / 2FA itself	That the request came from the account holder	Nothing - the account holder is the one being coached to authorize it

The common gap: **every existing layer checks the site, the code, or the account. Nothing checks the person's behaviour, in real time, on their own device, for their own benefit.**

3. Why This Project

This isn't a claim that behavioural fraud detection is a new field - enterprise vendors have sold session-based risk scoring to banks for years. The gap this project targets is specific: a **free, transparent, client-side, user-facing** tool, built for the individual rather than the institution. The scoring logic is plain, readable code anyone can audit - not a purchased black-box score a bank's fraud team reviews after money has already moved.

It is also an honest, scoped project: it targets the specific, common failure mode of a person being *rushed or pressured* into an action - not a promise to stop all fraud, which no single tool can do (see Limitations).

4. Objectives

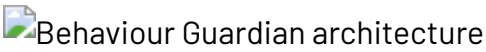
1. Detect, in real time and entirely on-device, behavioural patterns correlated with someone being rushed or pressured (panic typing, repeated corrections, repeated pasting into sensitive fields, frantic tab-switching, retried failed actions, and more).
2. Warn the person **in the moment**, not after the transaction has completed.
3. Do all of this **without ever reading** what was typed, what was pasted, or what the page contains - only event metadata.
4. Distinguish a **panicking human** from a **script/bot** driving the page, since the two need different responses and look almost opposite to each other.
5. Keep the scoring model **auditable and tunable**, not a black box - every warning should be traceable to specific, named reasons.
6. Fail safe toward the user's autonomy: this is a nudge, not a block - the person always keeps control of the final decision.

5. Solution – Behaviour Guardian

Behaviour Guardian is a Manifest V3 browser extension (plus a standalone, no-install demo of the same logic) that:

- Listens to page-level interaction *metadata* - keystrokes (not their content), paste events (not their content), focus changes, clicks, scrolling, tab visibility, and form submissions.
- Scores that activity against a transparent, hand-written rule set on a rolling time window, entirely inside the browser.
- Shows a calm, dismissible on-page warning when the pattern crosses a risk threshold - and separately flags when the pattern looks like automation (a script) rather than a panicking person.
- Never transmits anything off the device.

Architecture:




```
Page activity (keydown, paste, focus, click, scroll, submit, visibility)
        |
Content script  -- reads event *metadata* only, never values
        |
        +-- Risk engine (panic score)      -- sliding-window scoring
        +-- Automation heuristic (bot score) -- separate, opposite
signature
        |
        +--> Warning banner (on the page, only if panic risk is HIGH)
        +--> Background worker -- per-tab state, toolbar badge,
                  optional cross-tab "is a messaging app also open?" check
                  |
                  +--> Popup UI -- on-demand detail view + per-site opt-out
```

6. Complete Features (Detail)

6.1 Behavioural signal detection (the panic engine)

Signal	What it catches
Rapid, sustained typing	Continuous fast keystrokes in an 8-second window
High backspace rate	Repeated corrections - a hallmark of pressured typing
Multiple paste events	Several paste actions in a short span

Signal	What it catches
Repeated pasting into a sensitive field	Extra weight when those pastes specifically target an OTP/card/PIN-classified field
Focus churn	Rapid jumping between form fields
Field revisit	Re-focusing the *same* sensitive field three or more times (e.g. re-checking a UPI ID)
Panic clicking	Rapid, repeated clicks
Retry on the same control	Clicking the same "pay"/"submit" button repeatedly - a failed-payment retry pattern
Tab-switching	Frequent visibility changes, e.g. checking a message and coming back
Erratic scrolling	Rapid, large scroll jumps
Idle-then-burst	A long pause (waiting for instructions) immediately followed by a sudden flurry of activity
Fast-dwell submission	Submitting a sensitive form within seconds of arriving on the page
Rushed submission	Submitting right on top of any of the above bursts
Sensitive page context	The URL or the on-page fields look like a payment/login/OTP flow
Unfamiliar site (capped)	A small bonus if this is the first time the extension has seen this exact site - deliberately weak and never a trigger on its own

Core design rule: a single odd action never raises the risk above "looks calm." At least two distinct behavioural signals must fire together before anything is flagged - this is what stops a genuinely fast typist, or someone who mistyped one field, from being warned. This was verified empirically, not just designed on paper - see Section 12.

6.2 Automation / bot detection (a deliberately separate score)

A person panicking is erratic: bursts, corrections, hesitation. A script driving the page is the opposite: unnaturally *uniform* timing and clicks with no cursor movement beforehand. Folding both into one number would produce a score that means nothing, so this is tracked and displayed as a second, independent indicator - useful for contexts like checkout-bot abuse or exam autopilot detection, distinct from the scam-victim use case.

This heuristic is deliberately gated to real mouse input only (`pointerType === 'mouse'`) so keyboard-only and touch-only users - a

real accessibility population - are never mistaken for a bot.

6.3 The warning banner

A calm, non-blocking, dismissible banner injected into the page (via an isolated shadow DOM, so it can't be affected by the host page's CSS or vice versa) when the panic score reaches HIGH. It never stops the person from continuing - it names what was observed and gives them the option to pause.

6.4 Popup dashboard

Click-to-open toolbar popup showing: a live score and level for the current tab, a plain-language list of exactly which signals fired, a secondary automation indicator when relevant, and a "pulse strip" visual that reads as a flat calm line at low risk and a jagged spike at high risk.

6.5 Per-site opt-out

A toggle to stop monitoring on a specific site entirely, persisted locally so it survives across visits, for anyone who finds the tool unnecessary on a site they already trust.

6.6 Optional cross-tab correlation

A specifically-scoped, **off-by-default** feature: if the person opts in (a real browser permission prompt, not a silent grant), the extension can check whether a known messaging-app tab (WhatsApp Web, Telegram Web) is open at the same time as a rushed-behaviour episode - reflecting the common real-world pattern of a scammer coaching over one app while the victim acts on another. It only ever adds a small bonus on top of an already-detected behavioural pattern; it can never trigger a warning by itself.

6.7 Live, no-install demo

A single self-contained HTML file running the identical scoring logic, with three scripted playback buttons (calm user / panicked victim / bot) so the system can be evaluated without loading a browser extension.

7. User Roles

This is a single-user, client-side tool with no accounts or backend, so "roles" describe stakeholders rather than an access-control system:

- **End user (primary):** the person browsing, protected passively while doing nothing differently than usual.
- **Vulnerable user, protected indirectly:** e.g. an elderly relative or first-time internet user for whom a family member installs the extension - the direct beneficiary of the warning even if they didn't set it up themselves.
- **Reviewer / judge / evaluator:** uses the standalone demo to assess the system without needing to install anything.
- **Enterprise IT administrator (future scope, not implemented):** could push an org-wide install with adjusted thresholds via browser policy - see Future Scope.

8. Use Cases

Case 1 - WhatsApp "work from home" job scam. A message asks the victim to fill a Google Form and share an OTP "to verify eligibility." Behaviour: fast, pressured form-filling; repeated tab-switching back to WhatsApp; OTP pasted into an unfamiliar form. *Detected via:* focus churn, tab-switch, sensitive-paste-repeat, optionally the messaging-tab correlation.

Case 2 - Fake UPI payment screenshot. A scammer claims to have paid and asks the victim to "just check." The victim repeatedly re-enters and re-checks a UPI ID, retries a failed-looking payment, and pastes a number several times under pressure. *Detected via:* field revisit, retry-on-same-control, multi-paste.

Case 3 - Fake bank "customer care" call. A caller claims a card is blocked and talks the victim through their own, genuine bank login while staying on the phone. Rapid, coached OTP entry with corrections. *Detected via:* rapid typing, backspace rate, fast-dwell submission, sensitive context.

Case 4 - Marketplace refund scam (OLX/Flipkart-style). A "buyer" or "seller" sends a real-looking payment-collect link and pressures a quick

UPI approval. *Detected via:* idle-then-burst (waiting for the message, then acting fast), rushed submission.

Case 5 - Checkout bots / exam autopilot (automation use case). A script is driving form submission on a limited-drop product page, or an exam answer field is being filled by an automated tool rather than a person. *Detected via:* the separate automation heuristic - uniform keystroke timing, clicks with no preceding cursor movement.

9. Workflow (system / functional flow)

1. A page loads; the content script attaches its listeners and classifies whether the page looks sensitive (payment/login/OTP).
2. As the person interacts normally, every event is reduced to metadata (type + timestamp + coarse category) and pushed into rolling time windows inside the risk engine and the automation heuristic.
3. Roughly every 1.2 seconds, both engines recompute their current score from the retained window.
4. The result is sent to the background service worker for that tab only.
5. The background worker updates the toolbar badge (colour + score) and, if opted in, checks for a concurrently open messaging-app tab to apply a small bonus.
6. If the panic score reaches HIGH, a warning banner is injected directly into the page.
7. Opening the popup at any time shows the current tab's live state on demand.
8. A new page navigation clears the tab's state - each page load is a fresh behavioural episode by design (see Limitations, "no long-term profiling").

10. Business Flow (product / value delivery flow)

Stage	What happens	Value delivered
Discover	Person hears about the tool (word of mouth, a store listing, a relative recommends it)	Awareness
Install	Two clicks - load the extension, no account, no setup	Zero-friction adoption
Silent operation	Runs quietly during ordinary browsing; nothing to configure	No behaviour change required to benefit

Stage	What happens	Value delivered
Risk moment	Person is, in fact, being rushed on a sensitive page	The one moment the tool exists for
Nudge	A calm, specific, dismissible warning appears	A pause point where none existed before
Decision	The person - not the tool - decides what to do next	Preserves autonomy; this is assistive, not a block
Outcome	Best case: a scam is interrupted before money moves. Worst case: a false alarm the person dismisses in one click	Asymmetric payoff - the cost of a false positive is one click; the cost of a missed real one is a life savings

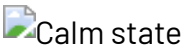
11. Before vs After

Aspect	Without Behaviour Guardian	With Behaviour Guardian
Who is protected	Only the website's own fraud team, after the fact	The individual, in the moment
Detection basis	Whether the *site* is safe	Whether the *person's current behaviour* looks pressured
Timing	Usually after money has moved (chargeback, bank fraud review)	Before the final submission, while there's still a choice
Visibility to the user	None - fraud scoring, where it exists, is invisible and server-side	A plain-language, on-page explanation of exactly what was observed
Data leaves the device	Depends on the tool; enterprise vendors' scores live on their servers	Never - everything happens on-device
Works on a genuine, unmodified site	N/A - this isn't what those tools check	Yes - this is precisely the case it's built for

12. Screenshots

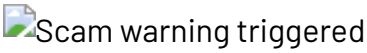
These are real, unedited screenshots captured from `demo/demo.html` running in headless Chromium, not mockups.

Calm state - normal interaction, no warning:

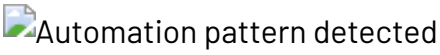

Calm state

Scam-victim simulation - reaches HIGH, banner shown. In the run that produced this screenshot, the score reached **100/100** with all of the

following genuinely firing together: fast typing, high backspace rate, multiple sensitive pastes, focus churn, field revisit, panic clicking, tab-switching, fast-dwell submission, and rushed submission.


Scam warning triggered

Bot / automation simulation - separate indicator, panic score stays low. In the same test run, the automation indicator reached HIGH while the panic score stayed at 30 ("Looks calm") - concrete evidence that the two engines behave independently, as designed, rather than one bleeding into the other.


Automation pattern detected

(Popup UI screenshots are not included here because the popup calls real `chrome.*` extension APIs that only exist inside a loaded browser extension context - they can't be faithfully rendered outside one. Load `extension/` as described in `SETUP.md` to see it live.)

13. Future Scope

- An options page exposing the currently-hardcoded thresholds as adjustable settings.
- Recalibrating the hand-tuned weights against real, consented usage data (or a small on-device classifier once such data exists).
- Publishing to the Chrome Web Store / Firefox Add-ons, including real icons and a formal privacy-policy page.
- Localized warning text - Hindi and other regional languages, given how much of the real-world scam pattern this targets happens in vernacular-language conversations.
- Enterprise/organizational deployment via browser policy, with administrator-configured thresholds.
- A broader, explicitly-consented messaging-app allowlist beyond WhatsApp Web and Telegram Web.
- A formal accessibility audit, including screen-reader testing of the warning banner.
- Cross-browser store verification (Firefox/Edge use the same WebExtensions APIs but haven't been independently re-tested there).

14. Limitations

- **The calm-victim blind spot.** This only helps when someone is *behaviourally* showing stress. A person who calmly and confidently enters a scammer's details because they're fully convinced won't trip any signal - no behavioural system, this one or an enterprise vendor's, can help there. This is stated plainly rather than hidden.
- **Hand-tuned thresholds, not a validated model.** The weights in `riskEngine.js` are a reasonable starting point chosen by design reasoning, not by training on labelled real-world sessions. Real-world false-positive and false-negative rates are currently unknown.
- **Per-tab, per-episode scope, by design.** There is no long-term behavioural profile of any person - each page load starts fresh. This is a deliberate privacy choice, not an oversight, but it also means the tool cannot catch a pattern that unfolds slowly across many separate sessions.
- **English-only warning text** in the current version.
- **Verified in Chromium only.** The WebExtensions APIs used are standard across Firefox and Edge, but cross-browser behaviour hasn't been independently re-tested.
- **It is a nudge, not a block**, by deliberate design choice - the person can always dismiss the warning and continue. This preserves autonomy but means the tool cannot force an outcome.
- **The optional messaging-tab check** currently recognizes only two named domains and requires an explicit permission grant to function at all.

15. Conclusion

Behaviour Guardian is a scoped, honest attempt at a real gap: giving an individual, in the moment, a client-side, transparent, plain-language warning when their own behaviour on a genuine website starts to look like someone being coached through a scam. It deliberately does not claim to solve fraud detection as a whole, does not compete with enterprise-grade site-safety tools, and does not claim to catch a calm, fully-convinced victim. What it does claim, and what running the demo verifies concretely rather than just asserting, is a narrower thing done carefully: detecting the specific, common, and previously unaddressed pattern of *rushed, pressured human behaviour on an otherwise safe page*

- and doing so without ever reading a single keystroke, password, or clipboard value.